

Algorithms for Data Science

Exercise 5

See the cheatsheet for valuable remarks!

For simplicity, in some of the tasks we consider only the keys of the key-value pairs.

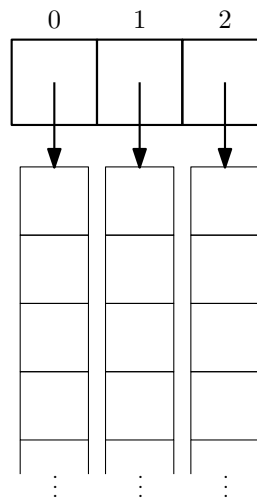
Task 5.1: Simple Tasks: Hash and BST

- (a) Add the following keys in the given order into a hash table of size 3 using the hash function

$$h(k) = k \mod 3$$

keys: 3, 5, 8

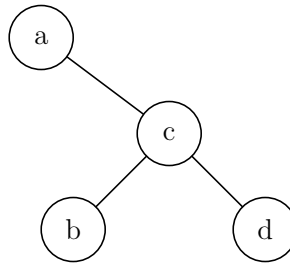
Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored.



- (b) Add the following keys in the given order to an initially empty binary search tree:

1, 3, 4, 2

- (c) Compute the height for each node and check whether the AVL-invariant is satisfied. *Recall that the height of a node v is the number of nodes on a longest path starting at v within the subtree of v .*



- (d) Design an algorithm `FindMin(root)` in pseudocode that returns the minimum key of the binary search tree rooted at `root` (whose root node is `root`). You can assume that the tree contains at least one element.
- (e) Draw a binary search tree that additionally satisfies the heap property with respect to the priorities containing the following key-priority pairs:

$(1, 1), (2, 2), (3, 3)$

The tree does not need to be a complete binary tree as required by our definition of a heap. It just have to be a binary tree that satisfies the BST invariant and heap invariant at the same time.

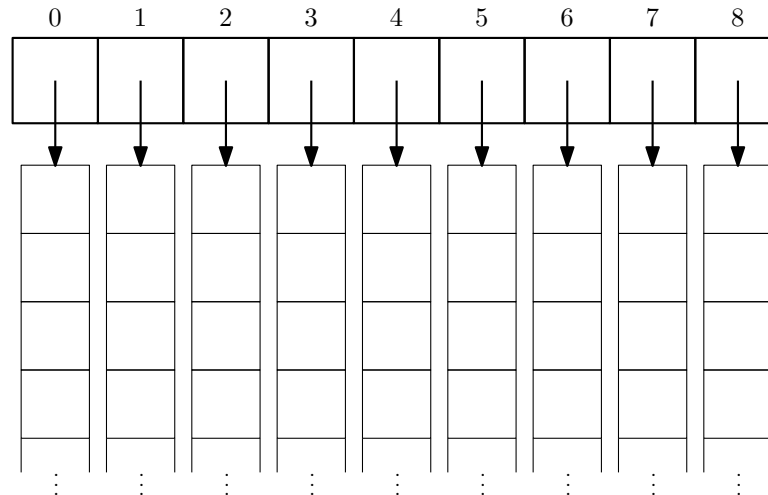
Task 5.2: Hash Tables

Add the following keys in the given order into a hash table of size 9 using the hash function

$$h(k) = k \bmod 9 .$$

keys: 5, 28, 19, 15, 20, 33, 12, 17, 10

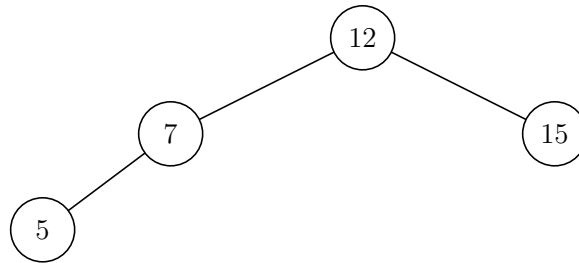
Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored.



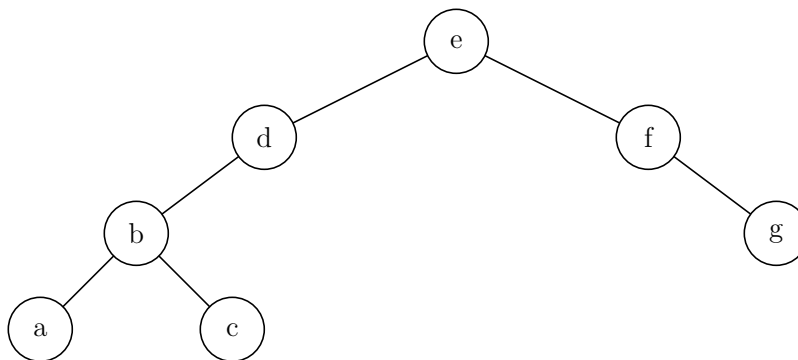
Task 5.3: Binary Search Trees

- (a) Given the binary search tree below, add the following keys in the given order:

10, 20, 8, 5, 2, 9, 11



- (b) Compute the height for each node and check whether the AVL-invariant is satisfied.



- (c) Design an algorithm **Add**(*root*, *k*, *v*) in pseudocode that adds the key-value pair (*k*, *v*) to the binary search tree rooted at *root*. (If the tree is empty, *root* = NONE.) The added node should be returned.

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 5.4: Which Data Structure?

For each of the following scenarios, propose the most appropriate data structure from our lecture (in terms of memory and time complexity). Explain your choice.

- (a) You want to keep track of all winning lottery numbers of a special lottery where the numbers have an unrealistic huge number of digits, say, several billions. You want to store each number together with its date, and you also want to check whether a given number has won in the past. The operations should be as fast as possible in practice and the memory usage not too large.

- (b) Upon registering on a social media platform, each user gets an automatic generated ID number. Your job is to maintain the data of all users that deleted their accounts. Since the data of deleted user accounts is (officially) kept only for legal reasons, the time to access and read such data does not matter. However the insertion time is very important as the platform wants to minimize the worst-case computational time (and thus the actual cost) spent on users that left.

Task 5.5: *Heap and BST*

- (a) The heap introduced in the lecture is actually called a *max*-heap, as its name-giving operations `FindMax()` and `RemoveMax()` are with respect to the largest element value inside the heap.

Design a *min*-heap using a max-heap as a black box! That is, implement a data structure that has only the following operations and that internally uses a max-heap:

- `Init()`: Initializes an empty min-heap.
- `Add(v)`: Adds the value v to the heap
- `FindMin()`: Returns the minimum value in the heap.
- `RemoveMin()`: Deletes an element with the minimum value in the heap and returns this value.

The running times should be asymptotically the same as the corresponding operations in the max-heap. You may assume that the elements are numbers and that the array underlying the max-heap is dynamic.

- (b) Draw a binary search tree that additionally satisfies the heap property with respect to the values containing the following key-value pairs:

$(3, 3), (2, 2), (4, 2), (10, 1), (5, 10)$

- (c) Design an algorithm `Successor(root, k)` in pseudocode that returns the smallest key that is larger than k among the keys of the binary search tree rooted at *root*. You can assume that k and its successor exist in the tree.
- (d) Design an algorithm `SortedOutput(root)` in pseudocode that prints all the keys of the binary search tree rooted at *root* in sorted order. The running time should be linear in the number of elements. If you visit a node *node*, you can print its key by calling `print(node.key)`.
- (e) Draw binary search trees of heights 3, 4, 5, 6 and 7 each containing the following keys:

2, 8, 10, 20, 26, 27, 31 .

Recall that the height of a tree is the number of nodes on a longest path from the root.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 5.6: *Interval Trees*

Design a data structure that maintains a dynamic set of intervals and allows the following operations, each running in $O(\log n)$ time, where n is the current number of intervals stored in the data structure.

1. `Search([a, b])`: returns true if your data structure contains the interval $[a, b]$, otherwise returns false.
2. `Insert([a, b])`: adds the interval $[a, b]$ to your data structure (if it does not exist yet).
3. `Delete([a, b])`: removes the interval $[a, b]$ from your data structure (if it exists).
4. `Intersects([a, b])`: checks whether your data structure contains at least one interval that has a non-empty intersection with the interval $[a, b]$.